

```
from collections import deque

# Rooms and their connections
graph = {
    'A': ['B', 'C'],
    'B': ['D'],
    'C': ['D', 'G'],
    'D': ['G'],
    'G': []
}

# BFS
def bfs(start, treasure):
    print("\n--- BFS Search ---")

    queue = deque([start])
    visited = set()

    while queue:
        room = queue.popleft()

        if room in visited:
            continue

        visited.add(room)
        print("Robot entered room", room)

        if room == treasure:
            print("Treasure found in room", room, "using BFS!")
            return

        for next_room in graph[room]:
            if next_room not in visited:
                queue.append(next_room)

    print("Treasure not found.")

# DFS
def dfs(start, treasure):
    print("\n--- DFS Search ---")

    stack = [start]
    visited = set()

    while stack:
        room = stack.pop()

        if room in visited:
            continue

        visited.add(room)
        print("Robot entered room", room)

        if room == treasure:
            print("Treasure found in room", room, "using DFS!")
            return

        for next_room in reversed(graph[room]):
            if next_room not in visited:
                stack.append(next_room)

    print("Treasure not found.")

# Main program
print("Hello! I am a Treasure Robot.")
print("Let's search for the treasure in rooms A, B, C, D and G.")

start = 'A'
treasure = 'G'

bfs(start, treasure)
dfs(start, treasure)
```

Hello! I am a Treasure Robot.
Let's search for the treasure in rooms A, B, C, D and G.

```
--- BFS Search ---
Robot entered room A
Robot entered room B
Robot entered room C
Robot entered room D
Robot entered room G
Treasure found in room G using BFS!
```

```
--- DFS Search ---
Robot entered room A
Robot entered room B
Robot entered room D
Robot entered room G
Treasure found in room G using DFS!
```

```
path1 = ['A', 'B', 'D', 'G']
path2 = ['A', 'C', 'G']

# Door costs
cost = {
    ('A', 'B'): 2,
    ('B', 'D'): 3,
    ('D', 'G'): 4,
    ('A', 'C'): 5,
    ('C', 'G'): 2
}

# Find cost of Path 1
cost1 = 0
for i in range(len(path1) - 1):
    cost1 = cost1 + cost[(path1[i], path1[i + 1])]

# Find cost of Path 2
cost2 = 0
for i in range(len(path2) - 1):
    cost2 = cost2 + cost[(path2[i], path2[i + 1])]

print("Path 1:", path1)
print("Total cost of Path 1:", cost1)

print("Path 2:", path2)
print("Total cost of Path 2:", cost2)

# Compare the costs
if cost1 < cost2:
    print("Path 1 is the cheapest path")
else:
    print("Path 2 is the cheapest path")
```

```
Path 1: ['A', 'B', 'D', 'G']
Total cost of Path 1: 9
Path 2: ['A', 'C', 'G']
Total cost of Path 2: 7
Path 2 is the cheapest path
```

